

EN3160 Assignment 2 on Fitting and Alignment

Ransika L.G.C. 220514C

1. Blob Detection using Laplacian of Gaussian and Scale-Space Extrema

- First, the image is blurred using a Gaussian filter. This reduces noise and smooths details. (The amount of blurring is controlled by the parameter σ)
- Next, we apply the Laplacian operator (which calculates second derivatives) to the smoothed image. (It detects rapid changes in intensity in a smoothed image)
- Instead of just one scale (σ), the algorithm repeats steps 1 and 2 for different values of σ this builds a scale-space cube.
- Blobs are identified as points that are local maxima (bright blobs) or minima (dark blobs) in the 3D scale-space cube (i.e., across position and scale).
- For each point (x,y,σ) , the algorithm checks if it has the highest response compared to its 26 neighbors (8 in 2D, plus neighbors in adjacent scales).

```
1 # Build LoG scale-space
2 h, w = gray.shape
3 log_cube = np.zeros((h, w, len(sigma_list)))
4 for i, sigma in enumerate(sigma_list):
5     gauss = gaussian_filter(gray, sigma)
6     log_img = laplace(gauss)
7     log_cube[:, :, i] = (sigma ** 2) * log_img # scale-normalized
8
9 # Non-maximum suppression in 3D scale-space
10 maxima = maximum_filter(log_cube, size=(3, 3, 3))
11 detected = (log_cube == maxima)
12
13 # Draw results: convert back to color image
14 draw_img = im_pil.convert('RGB')
15 draw = ImageDraw.Draw(draw_img)
16
17 for x, y, z in blobs_sorted[:30]:
18     sigma = sigma_list[z]
19     radius = int(np.sqrt(2) * sigma * scale_factor)
20     x_int, y_int = int(y), int(x)
21     bbox = [(x_int - radius, y_int - radius), (x_int + radius, y_int + radius)]
22     draw.ellipse(bbox, outline='blue', width=2)
```



2. Line and Circle fitting

(a): Line Fitting with RANSAC

How to Choose this Threshold

The assignment specifies noise with standard deviation $\sigma = 1.0$ for line points. For 95% confidence with Gaussian noise, $t = 1.96\sigma \approx 2.0$. For 99% confidence, $t = 2.58\sigma \approx 2.6$. I used a threshold of $t = 2.0$, which corresponds to capturing approximately 95% of true inliers.

```
1 def ransac_line(X, threshold=2.0, max_iterations=1000, min_inliers=40):
2     distances = compute_line_distances(X, a, b, d)
3     inlier_mask = distances < threshold # <-- ERROR THRESHOLD APPLIED HERE
4     n_inliers = np.sum(inlier_mask)
```

How to Choose Minimum Consensus

The dataset has 50 line points and 50 circle points (100 total). The expected inlier ratio is $w = \frac{50}{100} = 0.5$. The minimum consensus should be $d \approx (0.8 \text{ to } 1.0) \times w \times N$. For this dataset, $d \approx 0.8 \times 0.5 \times 100 = 40$. I used `min_inliers = 40` as the expected threshold.

```
1 for i in range(max_iterations):
2     # ... model fitting code ...
3
4     #CHECK IF CONSENSUS SET IS LARGE ENOUGH
5     if n_inliers > best_inliers: # <-- IMPLICITLY CHECKING CONSENSUS SIZE
6         best_inliers = n_inliers
7         best_params = (a, b, d)
8         best_mask = inlier_mask
9         best_sample = sample_points.copy()
```

(b): Circle Fitting with RANSAC

```
1 def compute_circle_radial_errors(X, center, radius):
2     distances_to_center = np.sqrt((X[:, 0] - center[0])**2 + (X[:, 1] - center[1])
3     **2)
4     radial_errors = np.abs(distances_to_center - radius) # <-- RADIAL ERROR
5     return radial_errors
```

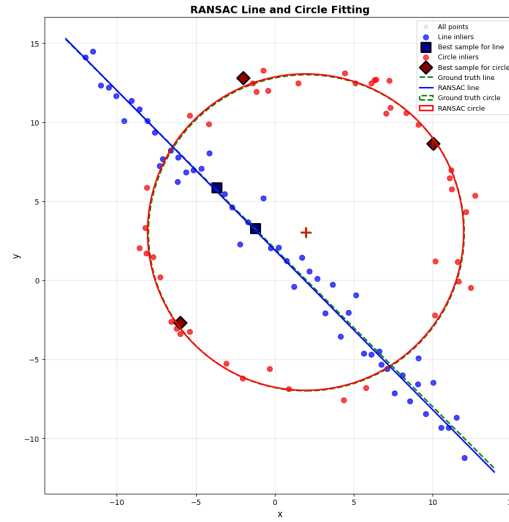
RANSAC Circle Parameters

Parameter	Location in Code	Value	Calculation / Description
Radial Error Threshold	<code>threshold</code> <code>ransac_circle()</code>	2.0	Based on circle noise $s = r/16 = 0.625$, using $t = 2.58\sigma \approx 1.61$ for 99% confidence.
Minimum Consensus	<code>min_inliers</code> <code>ransac_circle()</code>	40	$0.8 \times 50 = 40$ expected circle points.
Number of Iterations	<code>max_iterations</code>	1000	$N = \frac{\log(1-0.99)}{\log(1-(1-e)^m)}$, where $e \approx 0$.

(d) If the Circle is Fitted First Instead of the Line

- Higher outlier ratio:** Circle RANSAC faces $\sim 50\%$ outliers, requiring more iterations for robust estimation.
- Computational cost:** More iterations are needed for 99% confidence. The number of iterations $N = \frac{\log(1-p)}{\log(1-(1-e)^m)}$, where $p = 0.99$, $e = 0.5$, $m = 3$, giving $N \approx 35$ vs ~ 7 for line-first.
- Contaminated second fit:** Removing circle inliers leaves both line points and circle outliers, complicating line fitting.
- Sequential dependency:** The simpler line model (2 points) is easier to fit first, followed by the more complex circle (3 points); line fitting converges faster.
- Better separation:** Line-first removes a well-defined subset, leaving cleaner points for circle fitting.
- Conclusion:** Line-first is more efficient and provides better separation of the two geometric models.

(c): Visualization



3. Planar AR with homography

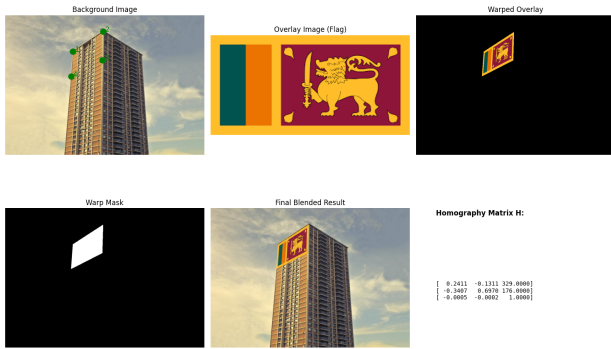


Figure 1: Running with CUSTOM implementation

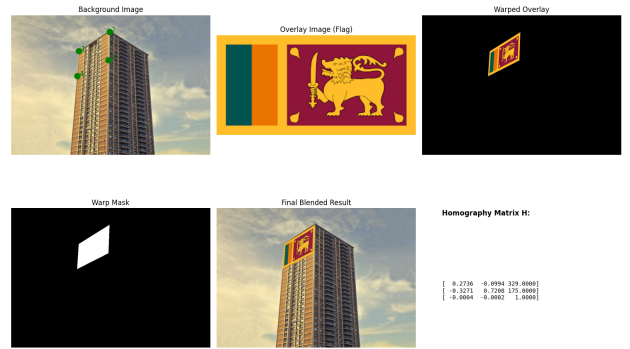


Figure 2: OPENCV implementation (for comparison)

Comparison of Homography Matrices

Custom Implementation:

$$\begin{bmatrix} 2.41113337 \times 10^{-1} & -1.31093150 \times 10^{-1} & 3.29000000 \times 10^2 \\ -3.4073039 \times 10^{-1} & 6.97019835 \times 10^{-1} & 1.76000000 \times 10^2 \\ -4.73299025 \times 10^{-4} & -2.31659765 \times 10^{-4} & 1.00000000 \times 10^0 \end{bmatrix}$$

OpenCV Implementation:

$$\begin{bmatrix} 2.73568824 \times 10^{-1} & -9.94385269 \times 10^{-2} & 3.29000000 \times 10^2 \\ -3.27078204 \times 10^{-1} & 7.20818230 \times 10^{-1} & 1.75000000 \times 10^2 \\ -4.11009879 \times 10^{-4} & -1.52042416 \times 10^{-4} & 1.00000000 \times 10^0 \end{bmatrix}$$

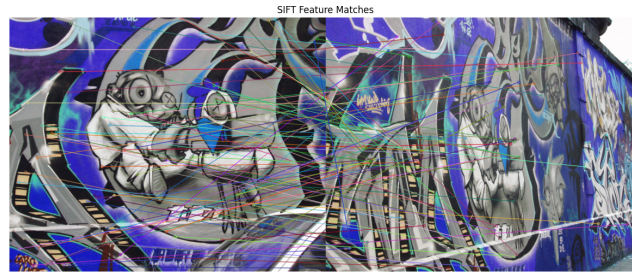
Difference:

$$\begin{bmatrix} 3.24554870 \times 10^{-2} & 3.16546233 \times 10^{-2} & 5.68434189 \times 10^{-14} \\ 1.36248343 \times 10^{-2} & 2.37983952 \times 10^{-2} & 1.00000000 \times 10^0 \\ 6.22891462 \times 10^{-5} & 7.96173494 \times 10^{-5} & 0.00000000 \times 10^0 \end{bmatrix}$$

Max difference: 1.0

4. SIFT Matching and Stitching

(a) Compute and Match SIFT Features Between the Two Images



(b) Homography Comparison

We have two homography matrices: one estimated using **RANSAC** and one from the **dataset**.
multicol

RANSAC Homography

$$H_{\text{RANSAC}} = \begin{bmatrix} -0.5983 & -0.6581 & 424.7572 \\ -0.4866 & -0.5406 & 348.7130 \\ -0.001394 & -0.001551 & 1 \end{bmatrix}$$

- Robust estimation removing outliers.
- Negative top-left 2×2 indicates rotation + reflection.
- Last row close to $[0 \ 0 \ 1]$.

Dataset Homography

$$H_{\text{dataset}} = \begin{bmatrix} 0.6254 & 0.0578 & 222.0122 \\ 0.2224 & 1.1652 & -25.6056 \\ 0.000492 & -0.0000365 & 1 \end{bmatrix}$$

- Computed without outlier rejection.
- Top-left 2×2 mostly positive: scaling + slight rotation.
- Smaller translation in second row.

Key Observations

Aspect	RANSAC	Dataset
Top-left 2×2	Negative $\rightarrow$ rotation + reflection	Mostly positive $\rightarrow$ scale + minor rotation
Translation	Large ($\approx 424, 348$)	Smaller ($\approx 222, -26$)
Robustness	Outlier-resistant	Sensitive to outliers
Last row	Close to $[0 \ 0 \ 1]$	Close to $[0 \ 0 \ 1]$

(c) Stitching Images

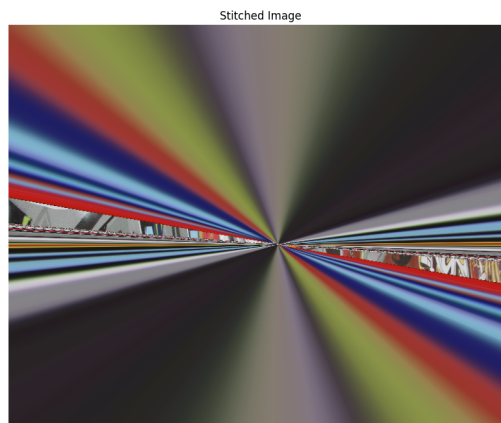


Figure 3: Stitched image result